Week 4: C++17 and SQLite Implementation

Lenka RS | Student number: 202321267

This pack follows the supplied seven-week schedule. It organises existing deliverables by their planned handover week; it does not invent weekly execution history or reviewer approval. Calendar dates are omitted from authored documents. Original machine-generated test evidence and Git provenance are retained.

Weekly handover checklist

Deliverable / checkpoint	Review action
Foundation code	Inspect role polymorphism, integer policy, repository port, prepared SQL and schema.
Integration plan	Connect services/UI in Week 5; code_reference is a selected excerpt, not a standalone build.
Code review	Check scope guards, ownership, parameter binding and transaction lifetimes.

7. Implementation and engineering decisions

The delivered source comprises three core translation units, a console composition root, typed headers and SQL migrations. Domain.cpp owns parsing, time formatting, role construction and the deterministic late policy. Services.cpp contains AuthService, AdminService and AssessmentService, keeping use-case orchestration separate from input prompts. SqliteRepository.cpp owns the prepared statements, relational mapping, schema creation, migrations and backup API. main.cpp converts console input into service calls and presents results. The shared public headers specify contracts without exposing SQL to the console. Consolidating small classes into these files is a documented implementation adjustment from the proposed one-file-per-service layout; it does not change the dependency direction.

The abstract User base has a virtual destructor and a pure virtual role method. Each concrete role overrides it, and makeUser returns unique ownership through the base type. Authorization invokes the virtual method, so the inheritance is exercised rather than included only for a marking criterion. It is deliberately a small model: role inheritance does not encode every resource permission. The application reloads the active account and relevant enrolment, teaching or department relationship. This supports revocation immediately on the next protected command and avoids trusting a menu or a stale session role as proof of authority.

An authenticated session is an opaque random token kept in AuthService memory. Its record contains user identity, last activity and a credential fingerprint. Password reset changes that fingerprint and invalidates the old session; account deactivation is checked afresh. Tokens disappear when the process ends. Failed-login counters and lockout deadlines persist in SQLite, preventing a process restart from clearing a five-minute lockout. A backwards clock movement invalidates a session rather than extending its lifetime. These measures do not defend against a privileged operating-system user altering the machine clock or database; that boundary belongs to workstation administration.

Passwords are hashed through libsodium's password-hash API, with algorithm metadata, salt and cost encoded in one stored string. Temporary credentials require change before normal commands. The terminal suppresses interactive password echo, and the console does not print stored hashes. Synthetic passwords in the tests are explicitly labelled test data. The system never attempts reversible password recovery: administrators reset credentials, and the audit identifies the action without storing the new plaintext. The benchmark uses a shared synthetic hash to construct its large fixture efficiently; separate security tests establish that independently created real account hashes have distinct salts.

Assessment mutations acquire an immediate SQLite transaction and re-evaluate policy before writing. A UnitOfWork object rolls back if an exception exits without commit. First marking writes a current assessment, updates submission phase and appends an event with NULL old marks. Subsequent edits preserve previous raw/final values and feedback. Moderation and release each produce their own revision. Expected row versions protect lecturer marking against changed submissions; expected assessment revisions protect departmental decisions against stale approvals. SQLite serializes writers, while these revision checks identify a logically outdated command even when it can acquire the database lock.

The persistence adapter binds user-supplied values into prepared statements. Table and query structures are fixed by implementation; no user string becomes SQL syntax. Foreign keys are enabled and verified for each connection, synchronous durability is configured, and a two-second busy timeout prevents an indefinite wait on another writer. Startup checks schema compatibility and runs quick_check, reporting damaged data rather than silently displaying an apparently empty school. An unsupported future schema is rejected. The program catches application and standard exceptions at the console boundary, emits a readable error and normally returns to the command loop. Fatal startup errors exit instead of continuing with an unusable database.

Work files are read before the write transaction, bounded at 10 MiB and stored as opaque BLOBs. The submitted path is reduced to a safe display filename. Storing bytes with the receipt avoids committing a record whose external attachment was never saved. Versions are immutable through normal SQL operations, and the latest version is derived rather than maintained as an independently editable pointer. This favors consistency over unlimited storage: repeated large submissions can grow the database significantly, so the custodian must monitor disk space and define retention outside the application. Work export writes a new destination without automatically opening untrusted content.

The implementation intentionally leaves the optional released-results CSV export out of scope. It includes authorized work export and consistent database backup, which serve different purposes. User roles and usernames are immutable after account creation; display names and activity status can change. A changed role requires a separately accountable account. There is no automatic missing-submission zero, because that would introduce a policy not defined in the agreed submission workflow. These limits keep the delivered Must requirements auditable and give future changes a clear starting point rather than an undocumented collection of partial features.

Supporting files

The accompanying folders contain the source references, manuals, scripts and real evidence listed in this pack. Read FILE_INDEX.md for exact paths.